System Design Notes: Caching

Oct 29, 2026 · @Salike Hassan

Caching basics

Caching doesn't mean "cache everything". Data is a good fit for caching only when all three hold:

1. It can be **served quickly**.
2. **Some staleness is acceptable**, so slightly old data is okay.
3. **Creating it is expensive** (heavy DB query, computation, external call).

Where do we cache?

The closer the cache is to the user, the faster and cheaper it is, but the harder it is to invalidate.

Browser → CDN → Gateway / API layer → Distributed cache (Redis) → DB

- **Browser:** we can't reach into the user's browser, so invalidation is manual (hard refresh, cache-busting).
- **CDN:** also manual. We have to upload or publish a new version of the file.
- **Gateway / API / Redis:** farther from the user, so easier to update and invalidate. Redis also supports distributed caching.

Cache-aside (the simple pattern)

1. Request comes in and we check the cache first.
2. **Cache hit:** return the data.
3. **Cache miss:** fetch from the DB, store it in the cache, then respond.

Updating the cache: 4 strategies

- **Read-through:** on a miss, the cache itself loads the data from the source, stores it and returns it. The app only talks to the cache. (In cache-aside, the app does the loading.)
- **Write-through:** on a write, update the cache and the DB together (synchronously), so both stay in sync.

- **Write-behind:** on a write, update the cache and respond immediately. The DB is updated later, asynchronously. Fast, but there is a risk of data loss if the cache fails before the flush.
- **Write-around:** on a write, go straight to the DB and skip the cache. The cache gets filled on the next read miss.

What doesn't qualify for caching

- Requests that are **unique every time**, where there is nothing to reuse.
- **Payments** (debit/credit, balance updates). They need **ACID** guarantees, so serving a stale or cached value is wrong. The cache is the wrong place for them.

Cache dependency risk

- Servers and teams get **addicted to the cache**, and the DB gets sized assuming the cache absorbs most of the traffic.
- If the cache goes down or runs cold, every request falls through to the DB. The DB is already at capacity, so it can't take it.

Distributed caching

Take Instagram: huge numbers of users, feeds and messages. Follower counts, profile pictures and some feeds are good cache candidates, but **one Redis node can't hold it all**. So the keys are spread across multiple Redis servers.

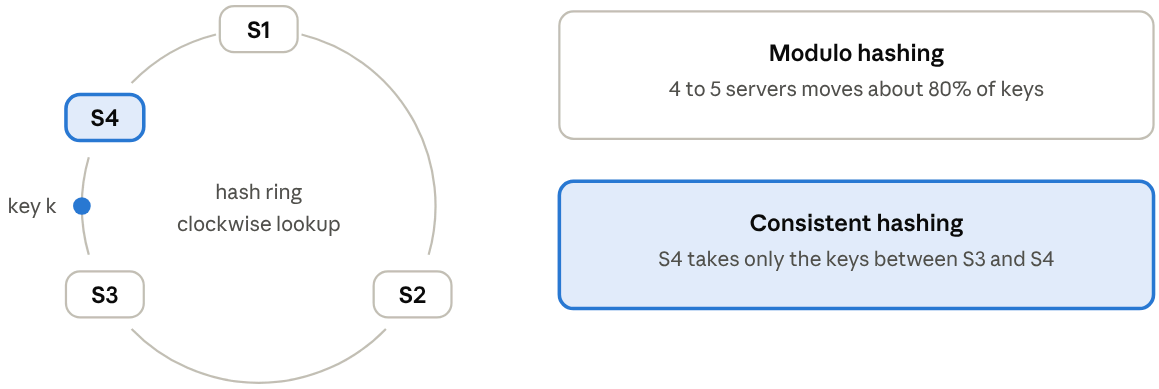
Modulo hashing (the naive way)

- With 4 servers, `hash(key) % 4` decides which server holds the key.
- Add a server and it becomes `% 5`. **About 80% of keys move**, only about 20% stay.
- The result is a flood of **cache misses**, and all of that traffic goes to the DB.

Consistent hashing

- Servers sit on a **ring**. A key goes to the **nearest server clockwise**.
- Add S4 and it only takes over the keys from its clockwise neighbour (S1 in the example). Everything else stays where it is.
- Only about **1/N of the keys** move, instead of about 80% with modulo hashing.

Adding S4 only takes keys from its clockwise neighbour, S1



Key k now lands on S4 instead of S1. Every other key stays where it was.

hash ring with S1 to S3 plus the new S4

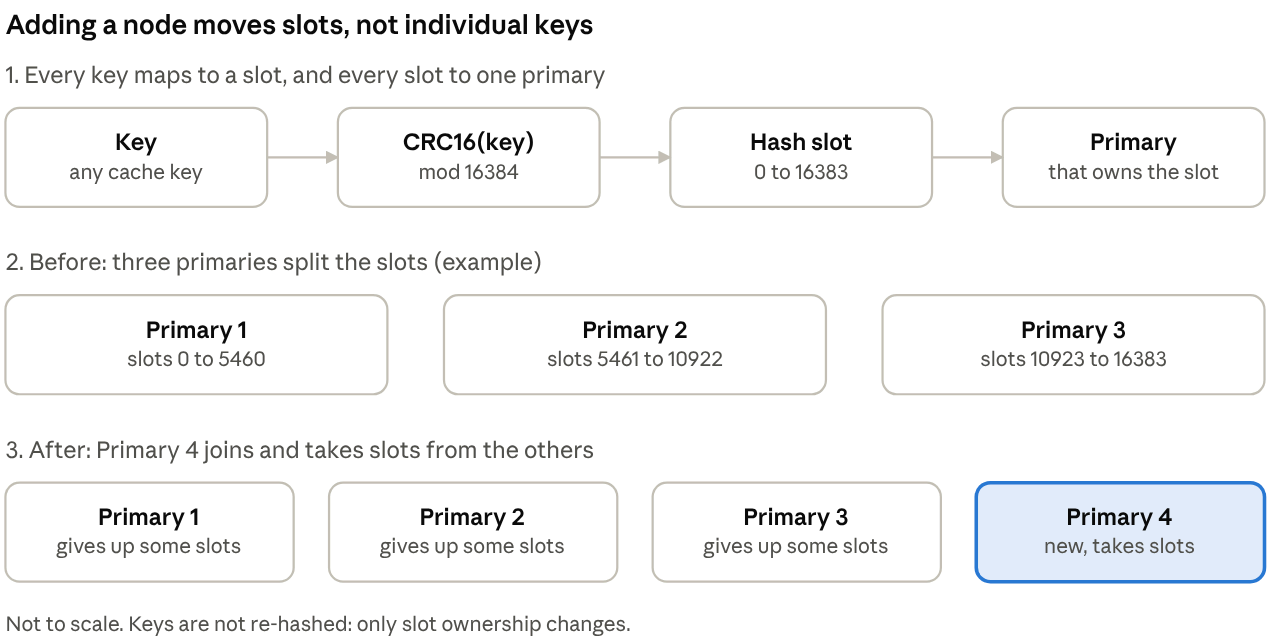
Adding S4 splits the arc S1 was serving. Only the keys in that arc change owner.

Fixed hash slots (Redis Cluster)

Redis Cluster doesn't place keys on a ring. It splits the key space into **16384 fixed slots** and gives each primary a range of them.

How it works

1. `slot = CRC16(key) mod 16384`.
2. The 16384 slots are divided into ranges across **primary 1 … primary N**.
3. A key lives on whichever primary owns its slot.
4. Every node knows the slot map. A client that asks the wrong node is redirected to the right one.



key to slot to primary, before and after adding Primary 4

A key's slot never changes. Only the slot-to-node mapping does.

When a new node is added

- The new primary joins with **no slots**, so it holds no keys yet.
- Slots are then moved from the existing primaries to it until they are spread evenly. With 4 primaries, each ends up with about a quarter.
- A slot's keys move with it. Keys are **not re-hashed**: key to slot stays the same, only slot to node changes.
- So only the keys in the moved slots shift (about 1/N of them), and slots can be migrated one at a time while the cluster keeps serving.
- Compared with the ring, slots add a fixed middle layer (key → slot → node), so rebalancing means moving whole slots between nodes.

Hot keys

- A celebrity, sportsman or politician profile gets far more visits than a normal user.
- Distribution doesn't help here: **one key still lives on one node**. That key keeps getting hit, stays in the cache, and overloads its node.
- **Fix:** keep the same key on **multiple servers** (the key plus a suffix, e.g. `profile:42#1`, `#2`, `#3`) and spread the reads across the copies.

- **Trade-off:** an update or invalidation now has to touch every copy.

Replicas and failover

- Every primary gets a **replica**.
- If a primary goes down, its replica is promoted. Without one, **all of that node's keys miss** and the traffic lands on the DB.
- Replication is asynchronous, so a few recent writes can be lost on failover. That is acceptable for a cache: most requests are still served from the replica and only a small part reaches the DB.

Expiration and TTL

A TTL is a **safety net** that stops stale data from hurting the business. It is **not an invalidation plan**.

How to decide the TTL

Ask three questions before setting it:

1. How harmful is it if the data stays stale?
2. How often does it change?
3. How costly is a miss?

What happens when a key expires?

It is not evicted right away. Two mechanisms work together:

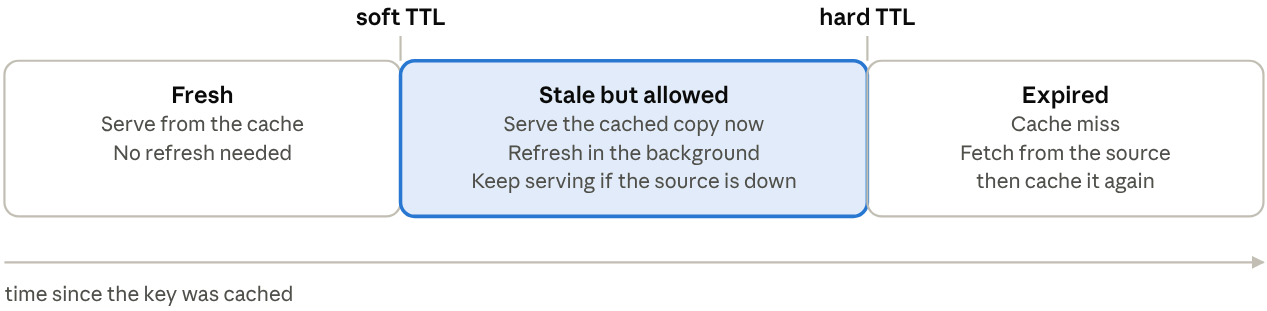
- **Passive (lazy) expiry:** the expired key just sits there until someone asks for it. On that request Redis sees it has expired and removes it. That is a miss, so the app serves from the source and caches it again.
- **Active expiry:** on a platform like Instagram we can't leave expired keys sitting in memory, so Redis also runs a background process(Reaper). It samples **20 random keys** from those that have a TTL and deletes the expired ones. If a large share was expired (more than 25%), it repeats. Its time per round is capped so it doesn't hurt Redis performance.
- **Memcached's version:** the **LRU crawler**. It uses a histogram of TTLs to decide what to scan and reclaim.

Avoiding a mass expiry

Cache 100 keys for exactly 1 hour and they all expire together. A burst of misses then lands on the DB.

- **Jitter:** add a random plus or minus of a few minutes to each TTL, so the keys expire at different times.
- **Soft TTL and hard TTL:**
  - Before the soft TTL: the data is fresh, serve it.
  - Between soft and hard: serve the stale data right away and **refresh it in the background**.
  - After the hard TTL: the key is gone. It's a miss, so fetch from the source and cache it again.
  - **Why it helps:** if the source has an outage, users still browse with slightly stale data instead of the whole app going down.

Between the soft and hard TTL, stale data is served while it refreshes



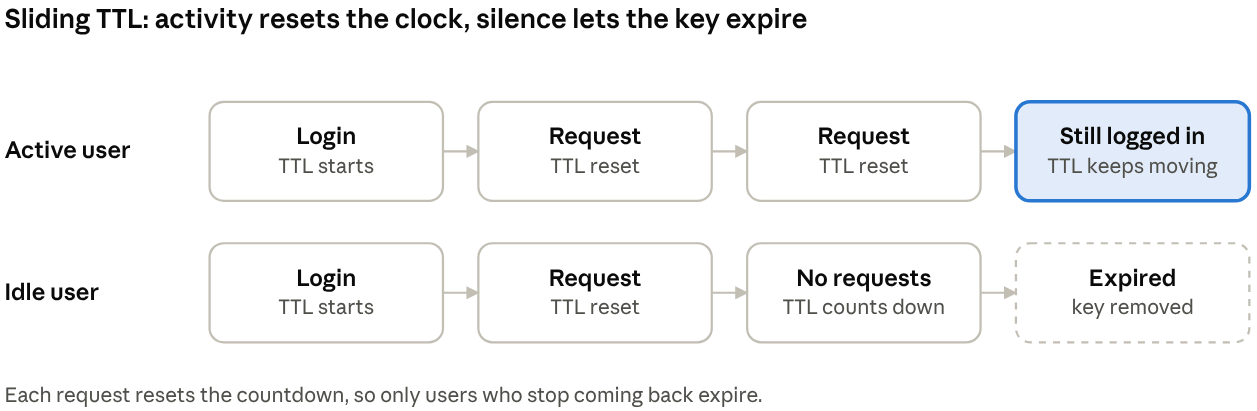
soft TTL and hard TTL zones, with what happens in each

Redis has only one expiry per key. So the soft deadline is stored inside the value, and the Redis TTL is set to the hard one. The same idea exists in HTTP caching as `stale-while-revalidate` and `stale-if-error`.

Sliding TTL

- Every time the key is used, its TTL is **reset**. Example: user sessions.
- Users who keep using the app keep their session. Users who stop are removed silently when the TTL runs out.
- Add a maximum total lifetime too, or an active session never ends.

- Not for data caches: a popular key would keep sliding and never refresh, so it would go stale.



sliding TTL for an active user and an idle user

Cache stampede

A stampede happens when a popular key (a celebrity profile) expires and many requests miss at the same moment. Each one rebuilds the same value from the DB.

Rough size of the problem: **duplicate rebuilds is about request rate x rebuild time**. At 1,000 requests per second and a 2 second rebuild, that is about 2,000 DB hits for one key.

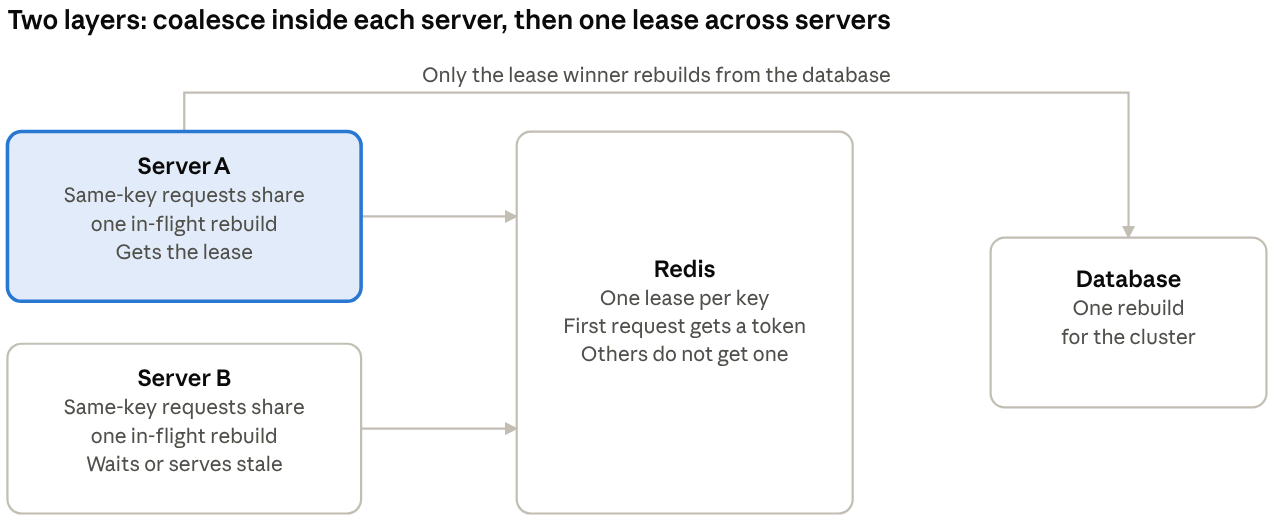
Jitter spreads different keys out, but one hot key still expires at a single instant. So we also have to control **who rebuilds**.

1. Lease

- On a miss, only **one request gets a lease**, a short-lived token that says: you rebuild this key. In Redis: `SET lock:profile:42 <random-token> NX PX 5000` (NX means only if not already set, PX is an automatic expiry).
- Everyone else does not touch the DB. They wait briefly and retry the cache, or serve the stale copy (this pairs with the soft TTL).
- The lease has its own short timeout, so a crashed rebuilder can't block the key forever.
- The idea comes from Facebook's memcache leases. The same token also stops stale writes (see the token section below).
- **Trade-off:** waiting requests add latency, and the lock costs one extra Redis round trip.

2. Coalesce per server

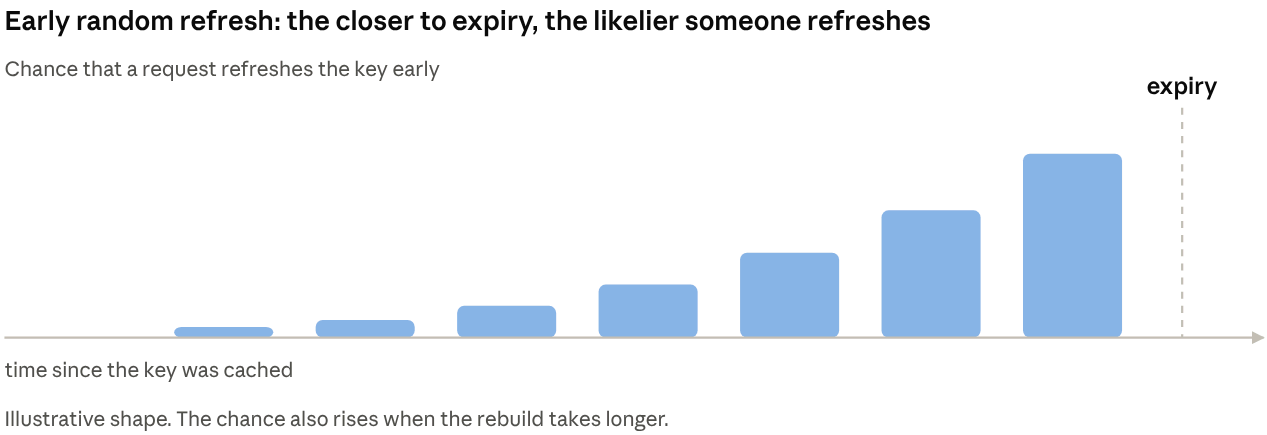
- Inside one app server, requests for the same key **share one in-flight rebuild**. The first request stores a promise (future) in an in-memory map by key. The rest await it. It is removed when done.
- No network call, so it is very cheap.
- It only works **per process**. With 50 servers you can still get 50 rebuilds.
- So combine the two: coalesce per server first, then take the lease across servers. That gives **one rebuild in total**. This is the same idea as Go's `singleflight`.



per-server coalescing plus a lease in Redis

3. Early random refresh

- Don't wait for the key to expire. A request that reads the key near its expiry has a small **random chance of refreshing it early**. The chance grows as expiry gets closer, so usually one request volunteers before the key expires and nobody sees a miss.
- The common rule (XFetch): refresh early if `now - delta * beta * ln(rand()) >= expiry`. `delta` is how long the last rebuild took (stored with the value), `beta` is about 1 (higher means more eager), and `rand()` is a random number between 0 and 1. A slower rebuild starts earlier.
- No lock and no coordination between servers.
- It is probabilistic, not a guarantee, so for a very hot key use it together with the lease.
- **Versus soft TTL:** a soft TTL is a fixed deadline, so a burst of requests can all trigger the refresh at once unless a lease or coalescing is in front. Early random refresh spreads the trigger out.

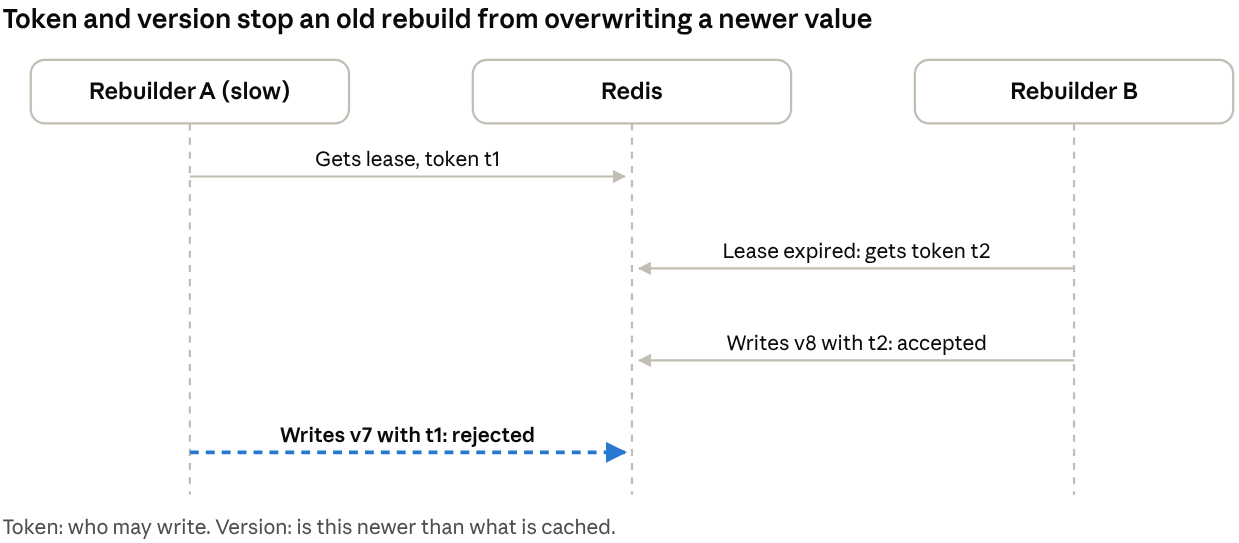


early random refresh, chance rises toward expiry

Concurrency in Redis: token and versioning

Even with a lease, rebuilders can race. A takes the lease but is slow. The lease expires, B takes a new lease and writes fresh data. Then A finishes and overwrites it with older data, or deletes B's lock.

- **Token:** the lease value is a **unique random token**. Only the holder with the matching token may release the lease or write. Releasing must be atomic compare-and-delete (a Lua script: if the lock equals my token, delete it). A stronger variant is a fencing token, an ever-increasing number from `INCR` that the write checks, so older holders are rejected.
- **Versioning:** every value carries a **version** (the DB row version or `updated_at`). A write is accepted only if its version is newer than the cached one, using compare-and-set in Lua or `WATCH` / `MULTI`. A slow, old rebuild is rejected. Another form is putting the version in the key (`profile:42:v17`). An update bumps the version, readers move to the new key, and old keys die by TTL.
- Use both: the token decides **who may rebuild**, the version decides **whether the result may overwrite**.



token and version rejecting a stale write

Libraries (singleflight equivalents)

Go's `golang.org/x/sync/singleflight` is the baseline. `Group.Do(key, fn)` allows one in-flight execution per key. Duplicate callers wait and share the result.

Language	Package	What it gives you
Python	`singleflight` (PyPI)	A port of Go's groupcache singleflight. Variants for threads, gevent and asyncio (`SingleFlight`, `SingleFlightGevent`, `SingleFlightAsync`). It only de-duplicates calls and has no cache, so wrap the cache-fill call.
Python	`oneflight` (PyPI)	Sync and async singleflight, decorators, and `forget(key)` to drop a stuck in-flight call. Python 3.10+.
Python	`dogpile.cache`	A cache library with a built-in lock so only one caller regenerates a value.

Language	Package	What it gives you
.NET	`HybridCache` (`Microsoft.Extensions.Caching.Hybrid`, .NET 9)	L1 memory plus L2 (Redis, SQL Server) with built-in stampede protection and tag-based invalidation.
.NET	`FusionCache` (`ZiggyCreatures.FusionCache`)	Stampede protection both local and distributed, fail-safe (reuse expired entries), soft and hard timeouts, eager refresh, jitter, and a backplane. It can also act as the HybridCache implementation.
.NET (DIY)	`ConcurrentDictionary<string, Lazy<Task<T>>>`	The per-server coalescing pattern by hand. No lease across servers.

- The Python packages coalesce **per process only** (technique 2). For the cross-server lease, use Redis `SET NX` with a token (redis-py has a `Lock` helper for this).
- In .NET, FusionCache already covers technique 2 and the cross-node part, and its fail-safe is our soft and hard TTL idea.

Cache failure and database protection

The cache hides load, and the app becomes addicted to it. With a **99% hit rate** and 100K requests per minute, only about **1,000** reach the DB. If Redis dies, about **100,000** do. That is **100x** the normal load. Even if the DB was sized with headroom for about 3,000 per minute, it now faces roughly 33x that, so it falls over. A wrong config pushed to the whole grid at once can cause exactly this.

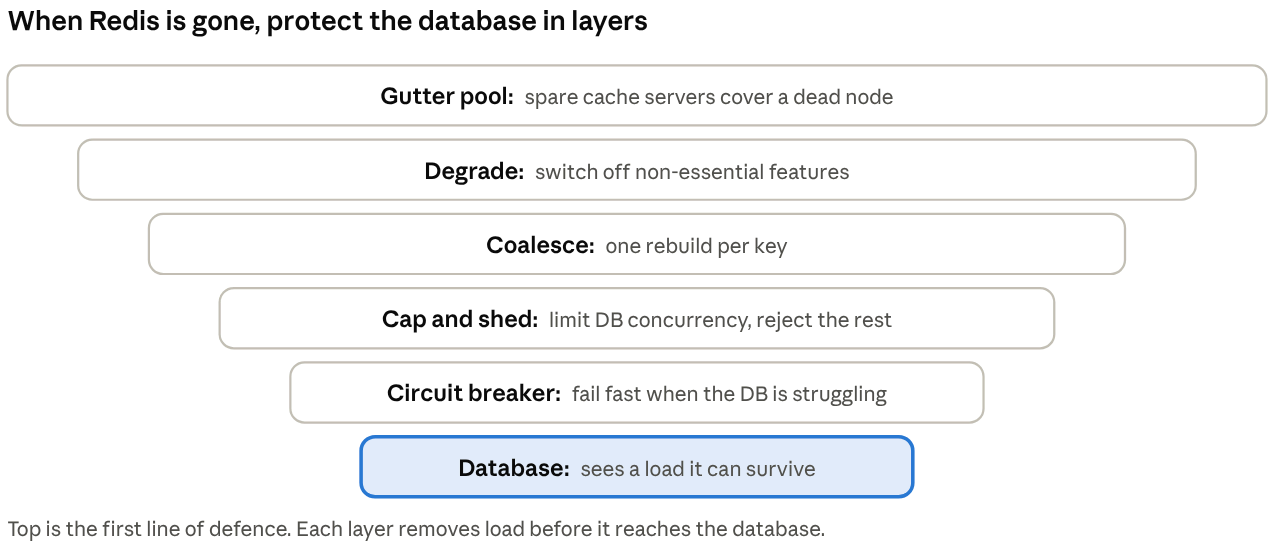
Protect the cache side

- **Replicas:** every primary has one (covered above).
- **Multiple availability zones:** put a primary and its replica in different zones, so one zone failing doesn't take both.
- **Gutter pool (Facebook's idea):** a small set of spare cache servers. When a cache node is down, its keys are routed to the gutter instead of the DB. Gutter entries get a **short TTL**, because invalidations don't reach the gutter, so brief staleness is fine.

- **Roll out config gradually:** canary a change on a small share first instead of changing the whole grid at once.
- **Bring the cache back gradually:** a restarted cache is cold, so everything misses and the DB gets flooded again. Send it a small share of traffic first (or restore from a snapshot), and raise the share as the hit rate recovers. Keep the stampede protection on while it warms up.

Protect the database side

- **Cap:** set a hard limit on concurrent DB requests and connections (a bulkhead). The DB only sees what it can take.
- **Circuit breaker:** if DB errors or latency pass a threshold, stop calling it for a short time and **fail fast** instead of piling on. Then let a few probe requests through (half-open) and close it again once they succeed.
- **Coalescing:** one rebuild per key (single flight and lease, covered above).
- **Load shedding:** when over the cap, **reject the excess early** (HTTP 503 or 429) instead of queueing. Reject the least important requests first, so the rest succeed.
- **Timeouts and backoff:** short timeouts, and retries with exponential backoff and jitter, so clients don't create a retry storm.



layers that protect the database when the cache is gone

Degrade gracefully

When Redis is down, keep the **essential** features and switch off the rest.

- **Keep on:** uploading a photo, posting a tweet, login, payments.

- **Turn off:** the For You feed (recommendations), view counts, like counts, other nice-to-haves that don't hurt the business.
- Use **feature flags or kill switches** that can be flipped without a deploy, and don't keep them in the Redis that just failed.
- Pre-decide what to show instead: a stale value, a default, or just hide the number.

Design and test it before the outage

During an outage the app is already hurting, so failover has to be **designed and agreed beforehand**.

- Run load tests **with the cache on and with it off**, and turn the cache off **in the middle of the run**.
- Test in steps: kill one node, then one availability zone, then the whole cluster. Watch DB load, latency, error rate and time to recover.
- Test the **recovery** too: bring the cache back cold and check that the gradual ramp keeps the DB safe.
- Practice it as a game day and keep a runbook. Start in staging, then run limited experiments in production with a way to abort.